

Software Verification

Master Programme in Computer Science

Mário Pereira `mjp.pereira@fct.unl.pt`

Nova School of Science and Technology, Portugal

September 28, 2026

Lecture 3

1. Presentation of the first handout

Functional Programming (3/3)

2. Algebraic specification and proof of an ADT
 - inductively defined propositions
 - verified Skew Heaps implementation

We have seen how to define recursive functions to manipulate lists:

```
Fixpoint length (l: list nat) : nat :=  
  match l with  
  | []  $\Rightarrow$  0  
  | _ :: r  $\Rightarrow$  1 + length r  
  end.
```

```
Fixpoint app (l1 l2: list nat) : list nat :=  
  match l1 with  
  | []  $\Rightarrow$  l2  
  | x :: r  $\Rightarrow$  x :: app r l2  
  end.
```

and how to make proofs about it:

```
Lemma app_length:  $\forall$ (l1 l2: list nat),  
  length (app l1 l2) = length l1 + length l2.  
Proof. induction l1. ...
```

We can also define **propositions** on lists:

```
Fixpoint In (a: nat) (l: list nat) : Prop :=  
  match l with  
  | []  $\Rightarrow$  False  
  | b :: m  $\Rightarrow$  b = a  $\vee$  In a m  
  end.
```

and use such propositions in proofs:

```
Lemma in_cons:  $\forall$ (a b: nat) (l: list nat),  
  In b l  $\rightarrow$  In b (a :: l).  
Proof. simpl. auto. Qed.
```

A common **higher-order function** on lists:

```
Fixpoint map {A B: Type} (f: A → B) (l: list A) : list B :=  
  match l with  
  | [] ⇒ []  
  | x :: xs ⇒ f x :: map f xs  
  end.
```

A property about such function:

```
Lemma in_map_iff: ∀[A B: Type] (f : A → B) (l : list A) (y : B),  
  In y (map f l) ↔ (∃ x : A, f x = y ∧ In x l)
```

```
Fixpoint Exists (P: nat → Prop) (l: list nat) : Prop :=  
  match l with  
  | [] ⇒ False  
  | x :: r ⇒ P x ∨ Exists P r  
  end
```

```
Fixpoint Forall (P: nat → Prop) (l: list nat) : Prop :=  
  match l with  
  | [] ⇒ True  
  | x :: r ⇒ P x ∧ Forall P r  
  end.
```

Let's take a more **logical** or **proof-oriented** approach:

$$\frac{(\text{EXISTS_CONS_HD}) \quad P \ x}{\text{Exists } P \ (x :: l)} \quad \frac{(\text{EXISTS_CONS_TL}) \quad \text{Exists } P \ l}{\text{Exists } P \ (x :: l)}$$

```

Inductive Exists : (nat → Prop) → list nat → Prop :=
| Exists_cons_hd : ∀(x : nat) (l : list nat),
  P x → Exists P (x :: l)
| Exists_cons_tl : ∀(x : nat) (l : list nat),
  Exists P l → Exists P (x :: l).
  
```

Here, we focus on the **derivation** of a **proof** that `Exists P l` holds.

$$\begin{array}{c}
 \text{(FORALL_NIL)} \\
 \hline
 \text{Forall } P \text{ []}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{(FORALL_CONS)} \\
 \frac{P \ x \quad \text{Forall } P \ l}{\text{Exists } P \ (x :: l)}
 \end{array}$$

Inductive Forall : (nat → Prop) → list nat → Prop :=
 | Forall_nil : Forall P []
 | Forall_cons : ∀(x : nat) (l : list nat),
 P x → Forall P l → Forall P (x :: l).

But is not always a matter of style!

```
Fixpoint sorted (l: list nat) : Prop :=  
  match l with  
  | [] => True  
  | [_] => True  
  | x :: y :: r => x ≤ r ∧ sorted (y :: r)  
  end.
```

Error:

Recursive definition of sorted is ill-formed.

Sorted as an Inductive Property

Inductive sorted : list nat → Prop :=

| sorted_nil: sorted []
| sorted_singleton: ∀x: nat, sorted [x]
| sorted_cons: ∀x y r,
 $x \leq y \rightarrow$
 sorted y :: r →
 sorted x :: y :: r.

$$\frac{}{\text{sorted } []} \qquad \frac{x : \text{nat}}{\text{sorted } [x]} \qquad \frac{x \leq y \quad \text{sorted } (y :: r)}{\text{sorted } (x :: y :: r)}$$

Verification of Abstract Data Types

ADT: Abstract Data Types

Main point: **abstract barrier**

- interface
- modularity
- reusability

The interface

```
type heap
```

```
val create : heap
```

```
val merge : heap -> heap -> heap
```

```
val add : nat -> heap -> heap
```

```
val remove_min : heap -> heap option
```

The interface + a specification

```
type heap
```

```
(* The data type definition *)
```

```
val create : heap
```

```
(* [create] returns a new, empty heap *)
```

```
val merge : heap -> heap -> heap
```

```
(* [merge h1 h2] takes two valid heaps [h1] and [h2]  
   and returns a valid heap, which is the result  
   of merging the elements of [h1] and [h2] *)
```

```
val add : nat -> heap -> heap
```

```
(* [add x h] creates a new valid heap, which  
   results from inserting [x] into the valid heap [h] *)
```

```
val remove_min : heap -> heap option
```

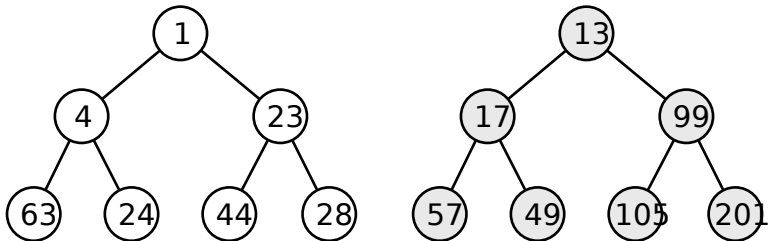
```
(* [remove_min h] removes the minimum element of [h],  
   if this is not the empty heap; otherwise returns [None] *)
```

Heaps Implementation

Possible implementation for the given interface: **Skew Heaps**.

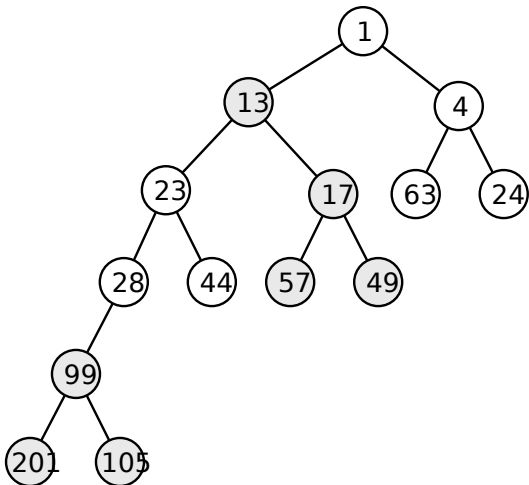
Skew Heaps encode heaps using simple **binary trees**.

Example:



The most important operation in any heap implementation: **merge**.

Example (after merge):



What does it mean for a heap implementation to be **correct**?

The **heap property**:

for any given node C ,
if P is a parent node of C ,
then the **key** (the value) of P
is less than or equal to the key of C .

Every **input** heap should respect the **heap property**.

Every **output** heap should respect the **heap property**.

Auxiliary definition `le_root`:

$$\frac{(\text{LE_ROOT_EMPTY}) \quad x : \text{nat}}{\text{le_root } x \text{ Empty}}$$

$$\frac{(\text{LE_ROOT_NODE}) \quad x \leq y}{\text{le_root } x \text{ (Node } l \ y \ r)}}$$

The main definition `is_heap`:

$$\frac{\frac{}{\text{is_heap Empty}} \quad (\text{IS_HEAP_EMPTY}) \quad (\text{IS_HEAP_NODE}) \quad \text{is_heap } l \quad \text{is_heap } r \quad \text{le_root } x \ l \quad \text{le_root } x \ r}{\text{is_heap (Node } l \ x \ r)}}$$

- Software Foundations, Volume 1:
`https://softwarefoundations.cis.upenn.edu/
lf-current/index.html`
(Chapters *Logic in Coq* and *Inductively Defined Propositions*)
- Chris Okasaki, *Purely Functional Data Structures*, Cambridge University Press, 1999.